

## **Relatório - Monitor de Processos em C**

Aluno: Rafael Santana

Disciplina: Sistemas Operacionais

Projeto: Monitor de Processos com Interação via API do SO

### **1. Breve explicação sobre chamadas ao sistema**

Chamadas ao sistema (system calls) são interfaces fornecidas pelo sistema operacional que permitem que programas em espaço de usuário interajam diretamente com os recursos de nível de kernel, como arquivos, processos, memória e dispositivos. No caso deste trabalho, utilizamos essas chamadas para acessar informações de processos em execução e para encerrar processos.

Essas interações foram realizadas principalmente por meio do sistema de arquivos virtual `"/proc"`, que é mantido dinamicamente pelo kernel e contém informações sobre todos os processos ativos.

### **2. Funções da API utilizadas**

`opendir()` / `readdir()` / `closedir()`: para percorrer os diretórios em `/proc` e identificar quais representam processos (baseando-se nos nomes numéricos).

`fopen()` / `fgets()` / `fclose()` / `sscanf()`: para abrir e ler os arquivos `/proc/[pid]/status`, de onde extraímos informações como nome do processo e memória utilizada (`VmRSS`).

`kill()`: chamada de sistema para enviar sinais para processos. Utilizamos `SIGTERM` para tentar encerrá-los.

`isdigit()` / `snprintf()` / `strcmp()`: funções auxiliares da biblioteca padrão para manipulação de strings e verificações.

`exit()`: para encerrar a execução do programa de forma limpa.

### **3. Capturas de tela (prints)**

As capturas de tela foram incluídas no vídeo de demonstração, evidenciando:

```
rafa@rafa-Latitude-3470: ~/Uneb/SO

21566      kworker/1:1-mm_percpu_wq          0 KB
21619      Web                             78332 KB
21679      kworker/u16:0-btrfs-endio-write    0 KB
21680      kworker/u16:1-events_unbound       0 KB
21681      kworker/u16:2-events_unbound       0 KB
21682      kworker/u16:3-btrfs-endio-write    0 KB
21698      Web                             77720 KB
21743      kworker/u16:4+events_unbound       0 KB
21861      kworker/2:2-events                0 KB
21877      kworker/0:2-events                0 KB
21965      kworker/1:2-mm_percpu_wq          0 KB
21990      kworker/u17:0                    0 KB
21998      kworker/3:0-events                0 KB
22017      Web                             77396 KB
22099      MonitordeProces                  1512 KB

--- Monitor de Processos ---
1. Listar processos
2. Encerrar processo (por PID)
3. Sair
Escolha: 2
Digite o PID do processo a encerrar: 21619
Processo 21619 encerrado com sucesso.
```

A listagem completa de processos com PID, nome e memória utilizada.

O sucesso ao encerrar um processo de nome "Web" após compilar e executar o programa com permissão de superusuário (foi dado um sudo MonitordeProcessos.exe).

#### 4. Comentários sobre dificuldades enfrentadas e o que foi aprendido

Durante o desenvolvimento do monitor, uma das principais dificuldades foi lidar com a permissão para encerramento de processos. Ao tentar encerrar um processo chamado "Web", recebi uma mensagem de erro indicando "Permission denied". Isso ocorreu porque o processo não era pertencente ao meu usuário e, por padrão, o Linux não permite que usuários comuns encerrem processos de outros.

Para resolver isso, optei por compilar o programa e executá-lo com permissão elevada, utilizando sudo no terminal. Isso me permitiu acessar e encerrar o processo com sucesso, demonstrando o funcionamento completo da aplicação.

Outra dificuldade menor foi entender como percorrer corretamente o diretório /proc e filtrar apenas os nomes numéricos que representam processos reais, o que foi resolvido com a função `is_number()`.